

Contents

Types of Security Controls.....	3
Security Policy and Components	4
Security Framework Design	5
Security Policy Framework (CIA Based)	5
Security Auditing Techniques.....	6
Cybersecurity Incident Analysis	6
Access Control Failure & Data Breach Prevention	7
Assurance	8
Trust	9
Process of Trust and Assurance (with Use Cases)	10
Secure and Trusted Systems	12
Use Case: Online Banking System — Building a Secure and Trusted System	13
Assurance in Requirement Definition and Analysis (SSDLC Phase 1)	16
Assurance During System and Software Design (SSDLC Phase 2).....	17
Formal Verification Technique	18
Formal Specification.....	19
Goals of Formal Evaluation	20
TCSEC (Trusted Computer System Evaluation Criteria)	21
International Efforts and ITSEC	22
Malicious Logic.....	23
1. Trojan Horses.....	23
2. Computer Viruses.....	24
3. Computer Worms.....	24
4. Other Malware	25
Vulnerability Analysis	26
Penetration Studies (Penetration Testing)	26
Vulnerability Classification	27
Auditing in Information Security.....	28
1. Anatomy of an Auditing System.....	28
2. Designing an Auditing System	28

3. A Posteriori Design	29
4. Auditing Mechanisms.....	30
Intrusion and IDPS (Intrusion Detection and Prevention Systems)	30
Intrusion Detection System (IDS)	31
Intrusion Prevention System (IPS).....	31
Difference Between IDS and IPS	31
Models of Intrusion Detection	32
Architecture of IDS/IPS.....	33
Organization of IDS.....	33
IPS Organization	33
Network Security	34
Network Policy Development	34
System Security Policy.....	34
User Security Policy.....	35
Program Security Policy	35
Operating System (OS) Security Policy and Architecture.....	35
Database Security Architecture	36

Types of Security Controls

Introduction

Security controls are mechanisms implemented to **reduce risk, protect assets, and enforce security policies**. They operate at different stages of an attack lifecycle and are essential for building a layered defense (**defense-in-depth**).

Classification Based on Function

1. Preventive Controls

Preventive controls aim to **stop security incidents before they occur** by reducing the attack surface.

Characteristics:

- Proactive in nature
- Focus on blocking unauthorized access
- Reduce probability of attack

Examples:

- Firewalls filtering unauthorized traffic
- Encryption protecting sensitive data
- Strong authentication mechanisms

2. Detective Controls

Detective controls identify and alert about **ongoing or past security incidents**.

Characteristics:

- Reactive in nature
- Provide visibility into system activity
- Enable incident detection

Examples:

- Intrusion Detection Systems (IDS)
- Log monitoring systems
- Security Information and Event Management (SIEM)

3. Corrective Controls

Corrective controls **restore systems after a security incident** and reduce impact.

Characteristics:

- Focus on recovery
- Minimize damage and downtime
- Support business continuity

Examples:

- Backup and restore systems
- Patch management
- Disaster recovery plans

4. Deterrent Controls

Discourage attackers by increasing perceived risk.

Characteristics:

- Psychological prevention
- Do not directly stop attacks but reduce likelihood

Examples:

- Warning banners
- Surveillance systems

5. Compensating Controls

Alternative controls used when primary controls are not feasible.

Characteristics:

- Flexible
- Used in constrained environments

Example:

Manual approval process when automated access control is unavailable

Example

In an enterprise network:

- Firewall (preventive)
- IDS (detective)
- Backup system (corrective)
- Warning banners (deterrent)

Security Policy and Components

Definition

A **security policy** is a formal document that defines **rules, responsibilities, and procedures for protecting organizational assets.**

Characteristics

- Comprehensive and enforceable
- Aligned with organizational goals
- Periodically reviewed and updated
- Covers technical and human aspects

Components

1. Policy Statement

Defines high-level security objectives and intent.

2. Standards

Mandatory controls specifying required configurations or behaviors.

3. Procedures

Detailed step-by-step instructions for implementing policies.

4. Guidelines

Recommended best practices that are flexible.

Additional Elements

- Roles and responsibilities
- Incident response plans

- Compliance requirements

Example

A corporate policy may:

- Require strong passwords (standard)
- Define password reset steps (procedure)
- Suggest password management practices (guideline)

Security Framework Design

Definition

A **security framework design** provides a structured methodology to **implement, manage, and continuously improve security controls**.

Characteristics

- Risk-driven approach
- Scalable and adaptable
- Aligns with industry standards
- Supports continuous improvement

Key Steps

1. Risk Assessment

Identify threats, vulnerabilities, and potential impacts.

2. Control Selection

Choose appropriate controls based on risk level.

3. Implementation

Deploy security mechanisms across systems.

4. Monitoring and Improvement

Continuously evaluate and update controls.

Example

An organization adopts **ISO 27001**:

- Identifies risks
- Implements controls
- Monitors compliance

Security Policy Framework (CIA Based)

Definition

A framework that aligns all security policies with **Confidentiality, Integrity, and Availability**.

Characteristics

- Balanced security approach
- Covers all aspects of data protection
- Widely accepted model

Components

1. Confidentiality Policy

Protects sensitive information from unauthorized access.

- Uses encryption and access control
- Limits data exposure

Example: Encrypting customer financial data

2. Integrity Policy

Ensures data accuracy and consistency.

- Uses hashing and validation
- Prevents unauthorized modification

Example: Verifying transaction data using hash values

3. Availability Policy

Ensures system resources are accessible when required.

- Uses redundancy and backups
- Prevents downtime

Example: Load-balanced cloud servers

Security Auditing Techniques

Definition

Methods used to **evaluate security controls, detect violations, and ensure compliance.**

Characteristics

- Continuous monitoring
- Supports accountability
- Provides forensic evidence

Techniques

1. Log Analysis

Review logs to detect anomalies.

2. Vulnerability Scanning

Identify known weaknesses.

3. Penetration Testing

Simulate attacks to test defenses.

4. Compliance Auditing

Ensure adherence to standards and policies.

Example

Analyzing login logs to detect brute-force attacks.

Cybersecurity Incident Analysis

Definition

Process of investigating security incidents to determine **cause, impact, and response strategies**.

Characteristics

- Structured and systematic
- Time-sensitive
- Supports future prevention

Steps

1. Detection

Identify abnormal activity.

2. Analysis

Determine root cause and impact.

3. Containment

Limit spread of attack.

4. Recovery

Restore normal operations.

5. Lessons Learned

Improve future defenses.

Example

Ransomware attack:

- Detect encrypted files
- Identify malware source
- Restore data from backup

Access Control Failure & Data Breach Prevention

Introduction

Failures in access control can lead to **data breaches**, where sensitive data is exposed to unauthorized users.

Causes of Access Control Failure

- Weak authentication
- Misconfigured permissions
- Lack of monitoring
- Insider threats

Characteristics of Data Breaches

- Unauthorized data access
- Data leakage or theft
- Financial and reputational damage

Prevention Techniques

1. Strong Authentication

Use multi-factor authentication.

2. Proper Authorization

Implement RBAC or MAC models.

3. Encryption

Protect data at rest and in transit.

4. Continuous Monitoring

Track user activities and detect anomalies.

5. Security Awareness

Educate users to prevent human errors.

Example

An organization:

- Implements MFA
- Restricts database access
- Encrypts sensitive data
 - Prevents unauthorized access and breaches

Conclusion

These topics collectively form the backbone of modern cybersecurity. From defining policies and frameworks to implementing controls, auditing systems, and preventing breaches, each aspect contributes to building a secure and resilient information system.

Assurance

Introduction

Assurance in information security refers to the **degree of confidence that a system correctly enforces its security policies and functions as intended without vulnerabilities**. It ensures that security mechanisms are properly implemented and are effective against threats.

Objectives of Assurance

- To verify that security controls are correctly implemented
- To ensure that the system behaves according to defined security policies
- To increase trust in the system's security mechanisms

Key Aspects of Assurance

1. Correctness

The system performs operations as intended without errors.

2. Effectiveness

Security controls successfully protect against threats and vulnerabilities.

3. Reliability

The system consistently maintains security under different conditions.

Methods of Assurance

1. Testing

Identifies vulnerabilities through security testing and penetration testing.

2. Formal Verification

Uses mathematical techniques to prove correctness of security mechanisms.

3. Evaluation

Independent assessment of system security against defined standards.

Example

In a banking system, assurance ensures that:

- transactions are processed correctly
- unauthorized access is prevented
- system remains secure under attack conditions

Trust

Introduction

Trust in information security refers to the **degree of confidence placed in a system, component, or entity to behave as expected and enforce security policies correctly**. A trusted entity is assumed to operate reliably and not violate security requirements.

Concept of Trust

Trust is an assumption that certain parts of a system will function correctly without malicious behavior. Security mechanisms rely on trusted components to enforce policies and maintain system protection.

Trusted Components

Examples of trusted entities include:

- **Operating System Kernel** – enforces access control and system security
- **Security Administrator** – manages permissions and policies
- **Trusted Computing Base (TCB)** – the set of components responsible for enforcing security

Trusted Computing Base (TCB)

The **TCB** consists of all hardware, software, and processes that are critical to system security.

Functions of TCB:

- Enforce access control policies
- Protect system resources
- Maintain system integrity

Characteristics of Trust

- **Reliability** – trusted components behave consistently
- **Correctness** – perform intended functions accurately
- **Security Enforcement** – enforce security policies without failure

Example

In a secure operating system:

- The kernel is trusted to enforce file permissions
- Users trust the system to prevent unauthorized access

Applications

- Operating system security
- Secure communication systems
- Digital certificate infrastructure

Process of Trust and Assurance (with Use Cases)

1. Trust Process — Use Case: Secure Operating System

Scenario

A **secure operating system** must ensure that only authorized users can access system resources and that security policies are enforced correctly.

Trust Establishment Process

Step 1: Identify Trusted Components

The system identifies components that must be trusted to enforce security.

Examples:

- OS Kernel
- Access control module
- Authentication system

Step 2: Define Trust Boundaries

Separate **trusted components** from untrusted ones.

User Applications → Untrusted

|

Operating System Kernel → Trusted

Step 3: Enforce Security through Trusted Components

Only trusted components (e.g., kernel) handle:

- Access control decisions
- Resource allocation
- Security enforcement

Step 4: Restrict Direct Access

Users and applications cannot directly access critical resources.

Example:

User → System Call → Kernel → Resource

Result

Security depends on trusting a **small, well-defined core (TCB)** rather than the entire system.

Application

- Operating systems (Linux, Windows)

- Secure kernels
- Embedded secure systems

2. Assurance Process — Use Case: Online Banking System

Scenario

An **online banking system** must ensure that transactions are secure, accurate, and protected from unauthorized access or manipulation.

Assurance Process

Step 1: Define Security Requirements

Identify what needs to be secured:

- Confidential transactions
- Accurate account balances
- Secure user authentication

Step 2: Design Security Controls

Implement mechanisms such as:

- Encryption (AES)
- Authentication (OTP, passwords)
- Access control policies

Step 3: Implement Security Mechanisms

Deploy security features in the system:

- Secure login system
- Transaction validation
- Secure database access

Step 4: Test the System

Perform:

- Vulnerability testing
- Penetration testing
- Security audits

Step 5: Evaluate and Verify

Check whether:

- Security policies are correctly enforced
- System behaves as expected

Step 6: Continuous Monitoring and Updates

- Monitor transactions
- Apply security patches

- Detect and respond to threats

Diagram

Security Requirements



Design Controls



Implementation



Testing



Evaluation



Monitoring

Result

Provides **confidence that the system is secure, reliable, and functioning correctly.**

Application

- Online banking systems
- E-commerce platforms
- Payment gateways

Key Difference in Application

Aspect	Trust	Assurance
Focus	Who/what is trusted	How security is verified
Approach	Assumption-based	Verification-based
Example	Trusting OS kernel	Testing banking system security

Secure and Trusted Systems

Introduction

A **secure system** is one that protects its resources and data from unauthorized access, modification, or disruption. A **trusted system** is one that is relied upon to correctly enforce security policies and behave as expected.

While all trusted systems must be secure, not all secure systems are necessarily trusted.

Secure System

A **secure system** implements security mechanisms to protect information and resources.

Characteristics:

- Protects confidentiality, integrity, and availability

- Uses mechanisms like encryption, authentication, and access control
- Defends against external and internal threats

Example:

A system using firewalls, encryption, and access controls to prevent unauthorized access.

Trusted System

A **trusted system** is a system that is **assumed to operate correctly and enforce security policies reliably**.

Characteristics:

- Enforces security policies correctly
- Contains a **Trusted Computing Base (TCB)**
- Provides assurance of correct behavior

Example:

An operating system kernel trusted to enforce access control rules.

Difference Between Secure and Trusted Systems

Parameter	Secure System	Trusted System
Definition	Protects system from threats	Reliably enforces security policies
Focus	Protection mechanisms	Correctness and reliability
Requirement	Implementation of controls	Assurance and trustworthiness
Example	Encrypted system	Verified secure OS kernel

Applications

- Operating systems
- Banking systems
- Military and government systems

Conclusion

A secure system protects against threats, while a trusted system ensures that security mechanisms function correctly and reliably. Both are essential for building dependable and robust information systems.

Use Case: Online Banking System — Building a Secure and Trusted System

Scenario

An **online banking system** allows users to perform transactions such as balance inquiry, fund transfer, and bill payments. The system must ensure **confidentiality, integrity, availability**, and also be **trusted to enforce security policies correctly**.

1. Define Security Requirements

Identify what needs protection:

- **Confidentiality** → Protect user credentials and transaction data
- **Integrity** → Ensure transaction amounts are not altered
- **Availability** → System must be accessible 24/7
- **Authentication & Authorization** → Only valid users perform operations

2. Design Secure Architecture

Separate system into controlled layers:

User → Web/App Interface → Application Server → Database

Security is enforced at each layer.

3. Implement Security Mechanisms (Secure System)

a) Authentication

- Username + password
- Multi-factor authentication (OTP)

b) Authorization

- Role-based access (customer, admin, auditor)

c) Confidentiality

- Encryption (AES for data storage, TLS for communication)

d) Integrity

- Hashing (SHA-256) for transaction verification
- Digital signatures for secure transactions

e) Availability

- Load balancing
- Backup servers
- Disaster recovery systems

4. Enforce Controlled Access (Trust Component)

Critical data should not be directly accessed by users.

User → Banking Application → Trusted Transaction System → Database

- Only **trusted programs** modify account balances
- Users cannot directly access or edit database

5. Establish Trusted Computing Base (TCB)

Define components that must be trusted:

- Operating system kernel
- Database management system
- Transaction processing system

These components enforce:

- Access control

- Data protection
- Security policies

6. Apply Security Assurance

Ensure the system is not just secure but also **verified**.

Steps:

- **Testing** → Penetration testing, vulnerability scans
- **Verification** → Validate transaction correctness
- **Evaluation** → Check compliance with security standards

7. Monitoring and Auditing

Continuously monitor system activities:

- Log all transactions
- Detect anomalies or fraud
- Perform regular audits

Example:

Transaction Log → Audit System → Detect Fraud

8. Ensure Separation of Duties

Prevent misuse by dividing responsibilities:

Role	Responsibility
User	Initiate transaction
System	Process transaction
Auditor	Verify transaction

Final Architecture Diagram

User

|

Authentication

|

Application Layer

|

Trusted Transaction System

|

Database

|

Audit & Monitoring

Result

A system becomes:

- **Secure** → Because it uses encryption, access control, and protection mechanisms

- **Trusted** → Because critical operations are handled by trusted components and verified through assurance processes

Assurance in Requirement Definition and Analysis (SSDLC Phase 1)

Introduction

In the **Secure Software Development Life Cycle (SSDLC)**, the **Requirement Definition and Analysis phase** is the first stage where security needs are identified. **Assurance in this phase** ensures that all security requirements are correctly defined, complete, and aligned with system objectives.

Objective

To ensure that security requirements are **accurate, complete, and capable of addressing potential threats and risks** before system design begins.

Key Activities

1. Identification of Assets and Security Goals

Critical assets such as data, applications, and system components are identified along with required security goals like confidentiality, integrity, and availability.

2. Threat and Risk Analysis

Potential threats and vulnerabilities are identified and analyzed to determine risks to the system.

Example:

- Unauthorized access
- Data breaches
- System failures

3. Definition of Security Requirements

Based on risk analysis, security requirements are specified.

Examples:

- Authentication mechanisms
- Access control policies
- Encryption requirements

4. Compliance and Policy Requirements

Ensure that system requirements comply with organizational policies and legal regulations.

5. Documentation and Validation

Security requirements are documented clearly and reviewed to ensure correctness and completeness.

Example

In an online banking system:

- Identify sensitive data (account details)
- Analyze threats (fraud, hacking)
- Define requirements (multi-factor authentication, encryption)

Importance

- Reduces design flaws and vulnerabilities
- Ensures security is built from the beginning
- Improves overall system reliability

Conclusion

Assurance in the requirement phase ensures that security needs are clearly defined and validated early in development, forming a strong foundation for building secure and trusted systems.

Assurance During System and Software Design (SSDLC Phase 2)

Introduction

In the **Secure Software Development Life Cycle (SSDLC)**, the **design phase** translates security requirements into a system architecture. **Assurance in this phase** ensures that the system design correctly incorporates security controls and is free from design-level vulnerabilities.

Objective

To ensure that the system architecture and design **properly implement security requirements** and provide a secure foundation for system development.

Key Activities

1. Secure Architecture Design

Design system structure with security in mind, including separation of components and controlled data flow.

Example:

- Layered architecture (User → Application → Database)
- Isolation of critical components

2. Selection of Security Mechanisms

Choose appropriate security controls based on requirements.

Examples:

- Encryption algorithms (AES, RSA)
- Authentication mechanisms (OTP, biometrics)
- Access control models (RBAC, MAC)

3. Threat Modeling

Analyze how attackers might exploit the system design and identify potential weaknesses.

Example:

- Identifying entry points for attacks
- Evaluating possible attack paths

4. Design Review and Validation

Review system design to ensure it meets all security requirements and does not introduce vulnerabilities.

5. Incorporation of Security Principles

Apply secure design principles such as:

- Least privilege
- Fail-safe defaults
- Complete mediation

Diagram

Security Requirements



System Design



Security Mechanisms



Design Review

Example

In an online banking system:

- Design secure login module
- Implement encrypted communication
- Restrict direct database access using trusted programs

Importance

- Prevents security flaws at early stage
- Reduces cost of fixing vulnerabilities later
- Ensures strong system architecture

Formal Verification Technique

Introduction

Formal verification is a method used to **mathematically prove that a system satisfies its security requirements and behaves correctly under all conditions**. Unlike testing, which checks selected cases, formal verification ensures correctness for **all possible inputs and states**.

Objective

- Ensure correctness of system design
- Detect logical errors and vulnerabilities
- Provide high assurance of system security

Concept

The system is represented using **mathematical models**, and its behavior is verified against formally defined security properties.

Techniques Used

1. Model Checking

System states are explored automatically to verify whether specified properties hold.

2. Theorem Proving

Mathematical proofs are used to verify that the system satisfies security properties.

Example

In a secure authentication system, formal verification can prove that:

- Unauthorized users cannot gain access
- Authentication rules are always enforced

Diagram

System Model

|

Formal Specification

|

Verification Process

|

Correctness Proof

Applications

- Cryptographic protocols
- Operating system kernels
- Safety-critical systems (defense, aviation)

Formal Specification

Introduction

Formal specification is the process of describing a system using **mathematical or logical expressions**. It defines system behavior and security requirements in a precise and unambiguous manner.

Objective

- Provide clear and precise system description
- Eliminate ambiguity in system design
- Serve as a basis for formal verification

Concept

Instead of using natural language, system requirements are written using **formal languages and mathematical models**.

Key Features

- Precision and clarity

- No ambiguity
- Enables verification and validation
- Supports automated analysis

Example

A formal specification may define access control as:

User can access file IF permission exists

This rule is expressed using logical expressions rather than informal text.

Diagram

System Requirements

|

Formal Specification

|

Mathematical Model

Applications

- Security protocol design
- Operating system design
- Database systems

Goals of Formal Evaluation

Introduction

Formal evaluation refers to the process of analyzing and verifying a system using formal methods to ensure that it meets defined security requirements.

Objectives / Goals

1. Ensure Correctness

Verify that the system behaves according to its specifications.

2. Detect Design Errors Early

Identify logical flaws and vulnerabilities during design phase.

3. Provide High Assurance

Increase confidence that the system is secure and reliable.

4. Validate Security Policies

Ensure that access control and security rules are correctly implemented.

5. Improve System Reliability

Reduce risk of system failure or security breaches.

Diagram

Formal Specification

↓

Formal Verification

↓

Formal Evaluation

↓

Assured System

Example

In a banking system, formal evaluation ensures:

- Transactions follow correct rules
- Unauthorized access is not possible
- Data integrity is maintained

TCSEC (Trusted Computer System Evaluation Criteria)

Introduction

TCSEC, also known as the **Orange Book**, is a security standard developed by the U.S. Department of Defense to evaluate the security of computer systems. It provides a framework for assessing how well systems enforce **confidentiality and access control**.

Objective

- Evaluate the security level of computer systems
- Ensure systems meet defined security requirements
- Provide assurance of system trustworthiness

Classification Levels

TCSEC defines security classes in increasing order of trust:

Class	Description
D	Minimal protection
C1	Discretionary Security Protection
C2	Controlled Access Protection
B1	Labeled Security Protection
B2	Structured Protection
B3	Security Domains
A1	Verified Design (highest level)

Key Features

- Focus on **confidentiality**
- Based on **Mandatory Access Control (MAC)**
- Emphasizes **Trusted Computing Base (TCB)**
- Requires system evaluation and certification

Example

A military system handling classified information may be evaluated under **B2 or A1 level** to ensure strict access control.

International Efforts and ITSEC

Introduction

With the global expansion of information systems, multiple countries developed their own security evaluation standards. To unify these efforts, international frameworks such as **ITSEC** and later **Common Criteria** were introduced.

ITSEC (Information Technology Security Evaluation Criteria)

ITSEC was developed by European countries to evaluate security of IT systems.

Objectives

- Provide flexible evaluation of security functions
- Support both **confidentiality and integrity**
- Separate functionality from assurance levels

Key Features

- Evaluates **security functions and assurance separately**
- Allows different levels of assurance (E0 to E6)
- More flexible compared to TCSEC

Comparison with TCSEC

Parameter	TCSEC	ITSEC
Origin	USA	Europe
Focus	Confidentiality	Confidentiality + Integrity
Flexibility	Less flexible	More flexible
Evaluation	Fixed classes	Separate assurance levels

International Security Standards

1. Common Criteria (CC)

Global standard combining TCSEC, ITSEC, and others.

- Defines Evaluation Assurance Levels (EAL1–EAL7)
- Used worldwide for security certification

2. ISO/IEC 27001

International standard for **Information Security Management Systems (ISMS)**.

- Focuses on organizational security policies
- Widely adopted across industries

Indian Standards and Efforts

1. IT Act, 2000 (India)

Legal framework for cybersecurity and electronic transactions.

2. CERT-In (Indian Computer Emergency Response Team)

- National agency for handling cybersecurity incidents
- Issues security guidelines and advisories

3. National Cyber Security Policy (NCSP)

- Provides framework for securing cyberspace in India
- Focuses on infrastructure protection and awareness

Malicious Logic

Introduction

Malicious logic refers to any **software or code intentionally designed to cause harm, disrupt operations, steal information, or gain unauthorized access** to systems. It is embedded within programs or transmitted through networks to exploit system vulnerabilities.

Characteristics of Malicious Logic

- **Intentional harm** – designed to damage or misuse systems
- **Hidden nature** – often disguised as legitimate software
- **Unauthorized execution** – runs without user knowledge or consent
- **Self-propagation (in some cases)** – spreads across systems or networks
- **Exploitation of vulnerabilities** – targets weak security controls

Types of Malicious Logic

1. Trojan Horses

Definition

A **Trojan Horse** is a malicious program that appears to be legitimate but performs **hidden harmful actions** when executed.

Characteristics

- Does not self-replicate
- Disguised as useful software
- Requires user execution

Working Principle

1. User downloads or installs seemingly legitimate software
2. Trojan executes in background
3. Performs malicious actions such as data theft or system compromise

Subtypes

- Backdoor Trojan
- Banking Trojan
- Remote Access Trojan (RAT)

Example

A fake software installer that secretly installs spyware on the system.

Mitigation

- Install software only from trusted sources
- Use antivirus software
- Avoid opening unknown attachments

2. Computer Viruses

Definition

A **computer virus** is a malicious program that attaches itself to a host file and **replicates when the infected file is executed**.

Characteristics

- Requires a host file
- Spreads through file execution
- Can modify or corrupt data

Working Principle

1. Virus attaches to a file or program
2. When the file is executed, virus activates
3. It replicates and infects other files

Subtypes

- File infector virus
- Boot sector virus
- Macro virus

Example

A virus that infects executable files and spreads when programs are run.

Mitigation

- Use updated antivirus software
- Avoid executing unknown files
- Regular system scanning

3. Computer Worms

Definition

A **computer worm** is a self-replicating malicious program that spreads across networks **without requiring a host file or user intervention**.

Characteristics

- Self-replicating
- Spreads automatically through networks
- Consumes system resources

Working Principle

1. Worm scans network for vulnerable systems
2. Exploits vulnerability
3. Replicates and spreads to other systems

Subtypes

- Email worms
- Network worms
- Internet worms

Example

Worm spreading through email attachments and infecting multiple systems.

Mitigation

- Apply security patches
- Use firewalls
- Disable unnecessary network services

4. Other Malware

Definition

Other malware includes various types of malicious software designed for specific harmful purposes.

Types and Characteristics

1. Spyware

- Monitors user activity
- Steals personal information

2. Ransomware

- Encrypts data and demands payment
- Prevents access to files

3. Adware

- Displays unwanted advertisements
- Tracks user behavior

4. Rootkits

- Hides malicious activities
- Provides unauthorized privileged access

Working Principle

- Installed through downloads or vulnerabilities
- Executes malicious actions such as data theft, system control, or disruption

Example

Ransomware encrypting files and demanding payment for decryption.

Mitigation

- Regular software updates
- Use antivirus and anti-malware tools

- Backup important data
- Avoid suspicious downloads

Vulnerability Analysis

Introduction

Vulnerability analysis is the process of identifying, evaluating, and prioritizing **weaknesses in a system that can be exploited by attackers**. It is a key activity in information security used to assess system security and reduce risks.

Objectives

- Identify system vulnerabilities
- Assess potential risks and impacts
- Provide recommendations to mitigate weaknesses
- Improve overall system security

Process of Vulnerability Analysis

1. **System Identification**
Identify system components such as hardware, software, and network infrastructure.
2. **Vulnerability Scanning**
Use tools and techniques to detect known vulnerabilities.
3. **Risk Assessment**
Evaluate the severity and impact of identified vulnerabilities.
4. **Reporting and Mitigation**
Document vulnerabilities and suggest corrective actions.

Penetration Studies (Penetration Testing)

Definition

Penetration testing is a controlled security testing method where security professionals simulate attacks to **exploit vulnerabilities and evaluate system defenses**.

Objectives

- Identify exploitable weaknesses
- Evaluate effectiveness of security controls
- Simulate real-world attack scenarios

Types of Penetration Testing

1. Black Box Testing

Tester has no prior knowledge of the system.

2. White Box Testing

Tester has full knowledge of system architecture and code.

3. Grey Box Testing

Tester has partial knowledge of the system.

Process

1. Information gathering

2. Scanning and enumeration
3. Exploitation of vulnerabilities
4. Reporting findings

Example

Testing a web application for vulnerabilities such as:

- SQL injection
- Cross-site scripting (XSS)

Importance

- Helps identify real attack paths
- Validates effectiveness of security measures

Vulnerability Classification

Introduction

Vulnerabilities are classified based on their nature and the type of system weakness they represent.

Types of Vulnerabilities

1. Design Vulnerabilities

Flaws in system architecture or design.

Example:

Improper access control design.

2. Implementation Vulnerabilities

Errors in coding or development.

Example:

Buffer overflow due to improper memory handling.

3. Configuration Vulnerabilities

Incorrect system or network configuration.

Example:

Open ports or weak firewall rules.

4. Operational Vulnerabilities

Weaknesses due to improper system use or maintenance.

Example:

Weak passwords or lack of updates.

Example

A web server with outdated software (configuration vulnerability) may be exploited by attackers.

Conclusion

Vulnerability analysis is essential for identifying and mitigating system weaknesses. Penetration testing provides practical validation of vulnerabilities, while classification helps in understanding their nature and prioritizing mitigation efforts.

Auditing in Information Security

Introduction

Auditing is the process of **monitoring, recording, and analyzing system activities** to ensure that security policies are being followed and to detect unauthorized or abnormal behavior. It plays a critical role in maintaining **accountability, detection of attacks, and system integrity**.

Objectives of Auditing

- Track user and system activities
- Detect security violations and intrusions
- Ensure compliance with security policies
- Provide evidence for investigation and analysis

1. Anatomy of an Auditing System

An auditing system consists of several components that work together to collect and analyze security-related information.

Components

1. Event Generator

Captures system events such as login attempts, file access, and network activity.

2. Audit Log

Stores recorded events in a structured format.

3. Analysis Engine

Processes audit logs to identify suspicious patterns or anomalies.

4. Response Mechanism

Triggers alerts or actions when abnormal activity is detected.

Diagram

Event Generator → Audit Log → Analysis Engine → Response

Example

A system records login attempts. If multiple failed logins occur, the analysis engine detects a possible brute-force attack and triggers an alert.

2. Designing an Auditing System

Key Considerations

1. What to Audit

Decide which events should be recorded:

- Login/logout activities
- File access
- Privileged operations

2. Level of Detail

Determine how much information to record:

- Minimal logging → less storage, less detail
- Detailed logging → more information but higher overhead

3. Storage and Protection of Logs

Audit logs must be securely stored and protected from tampering.

4. Performance Impact

Ensure auditing does not significantly degrade system performance.

5. Real-Time vs Batch Analysis

- Real-time → immediate detection
- Batch → periodic analysis

Example

In a banking system:

- All transactions are logged
- Logs are securely stored
- Alerts are generated for suspicious transactions

3. A Posteriori Design

Definition

A posteriori auditing design refers to analyzing system activities **after they have occurred**, rather than preventing them in advance.

Concept

- System allows operations to occur
- Audit logs record all actions
- Analysis is performed later to detect violations

Characteristics

- Focus on **detection rather than prevention**
- Useful for forensic analysis
- Helps identify attack patterns

Example

After a financial fraud incident, audit logs are analyzed to trace unauthorized transactions.

Diagram

System Activity → Audit Logs → Post Analysis → Detection

4. Auditing Mechanisms

Definition

Auditing mechanisms are techniques used to collect, store, and analyze audit data.

Types of Mechanisms

1. Logging Mechanism

Records system events such as access attempts and transactions.

2. Intrusion Detection Integration

Audit data is analyzed to detect suspicious patterns.

3. Real-Time Monitoring

Continuously monitors system activities and generates alerts.

4. Audit Trails

Maintains chronological records of user actions for accountability.

Example

- System logs record user actions
- Intrusion detection system analyzes logs
- Alerts are generated for suspicious behavior

Applications

- Banking systems (transaction tracking)
- Operating systems (user activity monitoring)
- Network security (traffic analysis)

Intrusion and IDPS (Intrusion Detection and Prevention Systems)

Introduction

An **intrusion** is any **unauthorized attempt to access, misuse, or disrupt a system or network**. To detect and respond to such activities, systems use **Intrusion Detection Systems (IDS)** and **Intrusion Prevention Systems (IPS)**. Together, they form **IDPS (Intrusion Detection and Prevention Systems)**.

- **IDS** → Detects suspicious activities and generates alerts
- **IPS** → Detects and actively blocks or prevents attacks

Concept of Intrusion

Definition

An **intrusion** is a violation of security policy where an attacker gains unauthorized access or performs malicious actions on a system.

Types of Intrusions

- Unauthorized access
- Privilege escalation
- Data theft
- System disruption

Intrusion Detection System (IDS)

Definition

An **IDS** is a system that monitors network or system activities to detect malicious behavior and security violations.

Types of IDS

1. Host-Based IDS (HIDS)

Monitors activities on a single system.

Example:

- File integrity checking
- Log monitoring

2. Network-Based IDS (NIDS)

Monitors network traffic for suspicious patterns.

Example:

- Detecting unusual packet activity

Working

1. Collect data (logs, network traffic)
2. Analyze data
3. Detect suspicious patterns
4. Generate alerts

Intrusion Prevention System (IPS)

Definition

An **IPS** is an advanced system that not only detects but also **prevents or blocks attacks in real-time**.

Functions of IPS

- Block malicious traffic
- Drop suspicious packets
- Reset connections
- Prevent exploitation of vulnerabilities

Difference Between IDS and IPS

Parameter	Intrusion Detection System (IDS)	Intrusion Prevention System (IPS)
Definition	A system that monitors and detects suspicious or malicious activities in a network or host.	A system that monitors, detects, and actively prevents malicious activities in real-time.
Primary Function	Detection and alert generation	Detection + prevention and blocking
Action Taken	Generates alerts/logs only	Blocks traffic, drops packets, resets connections

Mode of Operation	Passive (does not interfere with traffic flow)	Active (operates inline and controls traffic)
Placement in Network	Placed out-of-band (parallel to traffic flow)	Placed inline within the traffic path
Impact on Traffic	No direct impact on network traffic	Can delay or block traffic due to inspection
Response Speed	Slower (manual or delayed response)	Immediate response (real-time prevention)
Risk of False Positives	Less critical (alerts can be reviewed)	More critical (may block legitimate traffic)
Complexity	Relatively simpler to deploy	More complex due to inline operation
Control Capability	Cannot stop attacks directly	Can actively stop attacks
Example Action	Alert admin about suspicious login attempts	Block IP address after multiple failed attempts
Use Case	Monitoring and forensic analysis	Real-time protection of network systems
Dependency on Admin	Requires administrator intervention for action	Automatically takes action without admin intervention
Security Role	Detective control	Preventive + Detective control
Performance Impact	Minimal impact on performance	May affect performance due to deep inspection

Models of Intrusion Detection

1. Signature-Based Model

Detects attacks by comparing patterns with known attack signatures.

Example:

Detecting known malware signatures.

Limitation:

Cannot detect new (unknown) attacks.

2. Anomaly-Based Model

Detects deviations from normal system behavior.

Example:

Unusual login times or abnormal traffic patterns.

Advantage:

Can detect unknown attacks.

3. Specification-Based Model

Defines allowed behavior and detects violations of these rules.

Example:

System allows only specific operations; any deviation is flagged.

Architecture of IDS/IPS

Components

1. Data Collection Module

Collects system logs and network traffic.

2. Analysis Engine

Processes data to detect suspicious activities.

3. Knowledge Base

Stores attack signatures and rules.

4. Response Module

Generates alerts or blocks attacks.

Diagram

Data Collection → Analysis Engine → Knowledge Base → Response

Organization of IDS

IDS can be organized in different ways depending on system requirements.

1. Centralized IDS

All data is collected and analyzed at a central system.

2. Distributed IDS

Multiple IDS components are deployed across the network and share information.

3. Hybrid IDS

Combination of centralized and distributed systems.

Diagram

Distributed Sensors → Central Analyzer → Alerts

IPS Organization

IPS is placed **inline within the network** so that it can actively block malicious traffic.

Working

1. Traffic passes through IPS
2. IPS analyzes packets in real-time
3. Malicious packets are blocked
4. Alerts are generated

Diagram

Incoming Traffic → IPS → Allowed / Blocked → System

Applications of IDPS

- Network security monitoring
- Enterprise systems protection
- Cloud security
- Critical infrastructure protection

Network Security

Introduction

Network security refers to the set of policies, practices, and technologies used to **protect network infrastructure, data, and communication systems** from unauthorized access, misuse, modification, or disruption. It ensures secure transmission of data and protects systems connected through networks.

Objectives of Network Security

- Protect data during transmission
- Prevent unauthorized network access
- Ensure availability of network services
- Detect and respond to network attacks

Network Policy Development

Definition

Network policy development involves creating rules and guidelines that define how network resources are accessed, used, and protected.

Key Points

1. Defines acceptable use of network resources
2. Specifies access control rules for users and devices
3. Establishes security measures like firewalls and encryption
4. Includes monitoring and incident response policies

Example

A company policy that restricts external access to internal servers and allows only VPN-based secure connections.

System Security Policy

Definition

A **system security policy** defines security rules and controls applied to individual computer systems.

Key Points

1. Controls system access and user permissions
2. Defines authentication and authorization mechanisms
3. Specifies data protection requirements

4. Ensures system updates and patch management

Example

A server requiring administrator authentication for system configuration changes.

User Security Policy

Definition

A **user security policy** defines rules governing user behavior and responsibilities in maintaining system security.

Key Points

1. Defines password policies and authentication methods
2. Specifies acceptable user activities
3. Enforces user accountability through logging
4. Educates users on security best practices

Example

Users must use strong passwords and are prohibited from sharing credentials.

Program Security Policy

Definition

A **program security policy** defines security requirements for software applications and programs.

Key Points

1. Ensures secure coding practices
2. Restricts unauthorized program execution
3. Validates input data to prevent attacks
4. Controls software installation and updates

Example

An application validating user input to prevent SQL injection attacks.

Operating System (OS) Security Policy and Architecture

Definition

OS security policy and architecture define how the operating system enforces security controls and protects system resources.

Key Points

1. Implements access control mechanisms (DAC, MAC)
2. Enforces process isolation and memory protection
3. Uses authentication and privilege management
4. Maintains a Trusted Computing Base (TCB)

Example

Linux OS enforcing file permissions and user privilege separation.

OS Security Architecture Diagram

User

|

System Calls

|

Operating System Kernel (TCB)

|

Hardware Resources

Database Security Architecture

Definition

Database security architecture defines mechanisms to protect data stored in databases from unauthorized access and corruption.

Key Points

1. Implements access control for database users
2. Uses encryption to protect stored data
3. Maintains data integrity through constraints and checks
4. Provides auditing and logging of database activities

Example

A banking database restricting access so only authorized applications can modify account balances.

Diagram

User → Application → Database → Security Controls